CS 334: SQL Assignment

This assignment should be done **individually**. You may work with other people, get ideas, help each other debug, and so on; but the scripts that you submit should be your own.

Connecting to `lime`

PostgreSQL is installed on the department machine named `lime`. Make sure to connect to `lime` before doing anything else for the assignment. If you're connecting remotely via ssh from outside the building, you may have to connect to `skittles` first.

The Database

This assignment will use a database containing college-type data. The data is located in seven flat files. You can find all the flat files in this zip file.

Your first goal is to import this data into relations in PostgreSQL. The schema of the database that you should create is as follows (keys are in bold):

student (**sid**, sname, sex, age, year, gpa)
dept (**dname**, numphds)
prof (**pname**, dname)
course (**cno**, cname, **dname**)
major (**dname, sid**)
section (**dname, cno, sectno**, pname)
enroll (**sid**, grade, **dname, cno, sectno**)

To import a flat file into PostgreSQL, here are the steps you should follow:

1. Create a new directory for this assignment.
2. Copy all the data files into your new directory.
3. Start up PostgreSQL by typing **psql** at the command line. Enter in the password that you've been emailed.
4. Create a table in called **enroll**, which will contain the appropriate fields. I recommend that you put all your "CREATE TABLE" statements in a script called createtables.sql, which you can then invoke by typing **\i createtables.sql** .
5. Type **\copy enroll from enroll.data** into PostgreSQL to import the data. Don't put a semicolon at the end of the line.
6. Verify that the data made it into PostgreSQL correctly.

You should import the data from the other six relations similarly. The order that you import them in is relevant if you're using foreign keys.

Queries

Design SQL queries that answer the questions given below (one query per question) and run them using PostgreSQL. Your queries should be correct with respect to the design of this dataset.

In other words, we should be able to use your SQL queries on another dataset with the same schema and get correct answers even if the actual data within is different.

The query answers should be duplicate free, but you should use **distinct** only when necessary in general. Determining whether or not you need **distinct** should not depend on the particular dataset you have before you, but on its design.

The following questions are roughly in the order of degree of difficulty.

1. Print the names of professors who work in departments that have fewer than 50 PhD students.
2. Print the names of the students with the lowest GPA.
3. For each Computer Sciences class, print the class number, section number, and the average gpa of the students enrolled in the class.
4. Print the names and section numbers of all classes with more than six students enrolled in them.
5. Print the name(s) and sid(s) of the student(s) enrolled in the most classes.
6. Print the names of departments that have one or more majors who are under 18 years old.
7. Print the names and majors of students who are taking one of the College Geometry courses.
8. For those departments that have no major taking a College Geometry course print the department name and the number of PhD students in the department.
9. Print the names of students who are taking both a Computer Sciences course and a Mathematics course.
10. Print the age difference between the oldest and the youngest Computer Sciences major.
11. For each department that has one or more majors with a GPA under 1.0, print the name of the department and the average GPA of its majors.
12. Print the ids, names and GPAs of the students who are currently taking **all** the Civil Engineering courses.

All of your SQL queries should be contained in a single file called **queries.sql**.

Showing your results on a web page

An important application for database systems is supplying data for building web pages. Create a web page that will show the results from your first 3 queries above. There are many different technologies for doing this. For simplicity, we're going to embed SQL within a PHP script. Here's how do so:

1. On `lime`, **cd** to the directory **/Accounts/courses/cs334/web-directories/[username]**, where **[username]** indicates that you should use your usual username.
2. Copy into this directory the sample web page that I have created, which you can find at **/Accounts/courses/cs334/web-directories/dmusican/index.php**. Since these directories all contain web pages, they are publicly readable.
3. Modify this PHP script to display the results of your first three queries.
4. View your results in web browser by visiting **http://cs.carleton.edu/cs334/[username]/index.php** .

Remember that these scripts are readable by anyone in the class! That's why I've asked you to put the SQL for your first 3 queries only on the web page. Grading the web page will not be a significant portion of the assignment, but is intended to be a fun way for you to see how your results could be used.

Multiple Parts

Part 1: Submit queries 1-4.

Part 2: Submit queries 5-12 and the web page.

What to Turn In

Submit via Moodle your **queries.sql** as well as a copy of your **index.php** file (for part 2).

Bonus questions (no extra credit, but more to do if you're having fun)

1. Does the dataset that I've given you allow one to "cheat" at any of the above questions, i.e. issue a query that is wrong with respect to the schema but still gives the correct answers for this particular instance? If so, which questions are they, and what is it about this sample instance that allows "wrong" queries to work?
2. There's a programming framework called Django that's quite popular; it allows you to query a database entirely via Python objects, without embedding SQL within your code. It is amazingly cool for developing web applications quickly, and for writing straightforward queries. Writing complex queries in Django, however, can be more complex than in SQL or not doable at all. (Django lets you write a "raw" SQL query if you like, but if you're doing a lot of this, you're losing most of the benefit of using Django in the first place.) If you've been enjoying working with Django, or if you want to experiment with it, install it yourself and see if you can figure out how to rewrite the above queries without using raw SQL.